RAPPORT DE MI-PROJET

Rapport C++

Structures de données et affichage 3D

Réalisé par

Raïan CHOUIREF

Dorian LEON FERNANDES

Année universitaire : 2026–2027

16 octobre 2026

Table des matières

1 Introduction 1

2 Problème 1

3 Contexte du projet 2

4 UML 3

5 Loader 6

6 Halfedge 7

7 Affichage 8

8 Conclusion 10

1. Introduction

Ce projet consiste à développer un éditeur 3D capable de charger, afficher et manipuler des modèles 3D à partir de fichiers aux formats .PLY et .OBJ. L'objectif principal est de permettre à l'utilisateur d'interagir avec les modèles 3D de manière intuitive et efficace. Pour cela, nous devons mettre en place une structure de données adaptée pour représenter les maillages 3D, ainsi qu'un système d'affichage performant utilisant OpenGL. Les grandes étapes du projet incluent la conception de la structure de données Halfedge, le développement du chargeur de fichiers, l'implémentation du système d'affichage et la création d'une interface utilisateur.

2. Problème

Dans ce projet, nous sommes confrontés à la problématique de la manipulation et de l'affichage de modèles 3D à partir de fichiers aux formats .PLY et .OBJ. L'objectif principal est de développer un éditeur 3D capable de charger ces fichiers, d'afficher les modèles correspondants et de permettre à l'utilisateur d'interagir avec eux. Les objectifs spécifiques du projet incluent :

- La mise en place d'une structure de données efficace pour représenter les maillages 3D, notamment en utilisant la structure Halfedge.
- Le développement d'un système d'affichage utilisant OpenGL pour rendre les modèles 3D de manière réaliste et performante.
- La création d'un chargeur de fichiers capable de lire les formats .PLY et .OBJ, d'extraire les informations géométriques et de les convertir en structures de données utilisables par le programme.

3. Contexte du projet

Dans le cadre de la matière Computer Science, nous avons été amenés à réaliser un projet en C++ portant sur la programmation et la géométrie. L'objectif principal de ce projet était de développer un éditeur 3D capable de charger, afficher et manipuler des modèles 3D à partir de fichiers aux formats .PLY et .OBJ. Pour ce faire, nous avons utilisé plusieurs outils et bibliothèques, notamment OpenGL pour le rendu graphique, ainsi que des structures de données adaptées pour représenter les maillages 3D. Les contraintes du projet incluaient la gestion efficace des données géométriques, la mise en place d'une interface utilisateur intuitive et la compatibilité avec différents formats de fichiers 3D. Nous avons également dû nous assurer que notre éditeur 3D soit capable de gérer des modèles complexes tout en maintenant des performances optimales.

Pour le moment, nous avons réussi à implémenter les fonctionnalités de base, le chargement des fichiers .PLY et .OBJ, ainsi que l'affichage des modèles 3D. Cependant, il reste encore des améliorations à apporter, notamment en ce qui concerne l'optimisation des performances et l'ajout de fonctionnalités supplémentaires pour enrichir l'expérience utilisateur.

4. UML

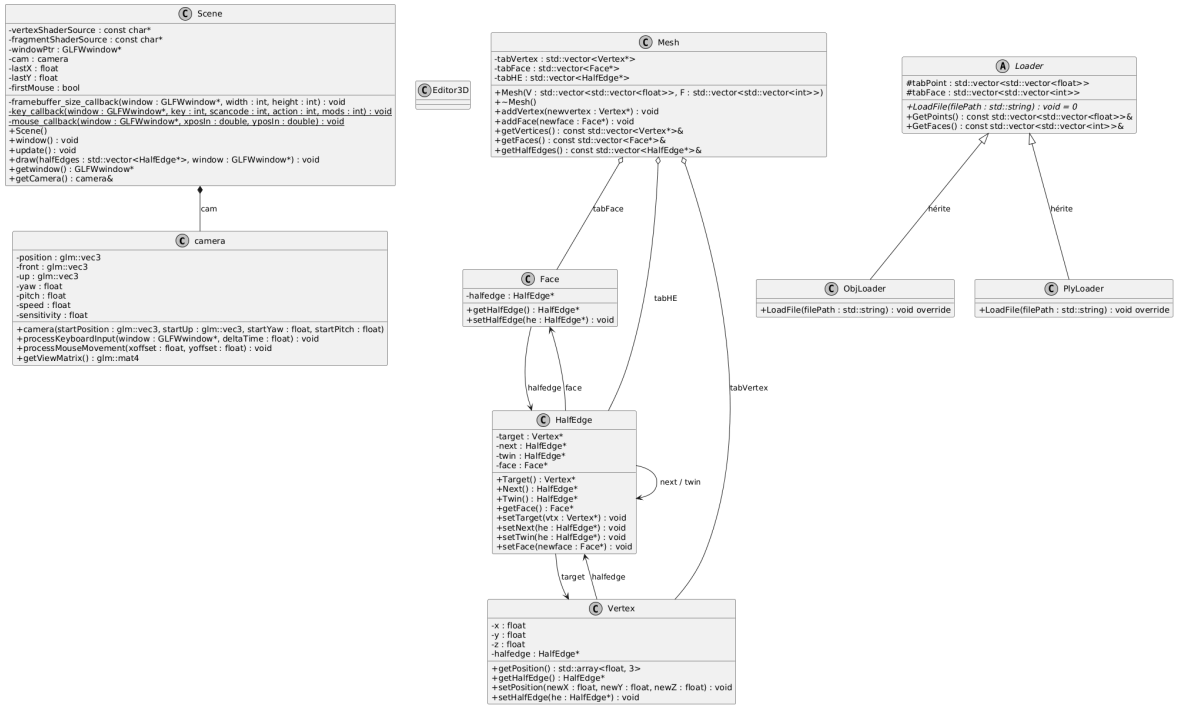


FIGURE 1 – Diagramme UML du projet

Nous avons utilisé un diagramme UML pour représenter les classes et leurs relations dans notre projet. Le diagramme montre les principales classes, leurs attributs et méthodes, ainsi que les associations entre elles. Cela nous a permis de mieux comprendre la structure du projet et de faciliter la communication entre les membres de l'équipe.

Voici une explication des principales classes et de leurs relations :

- La classe `Scene` représente la scène 3D et contient des objets tels que des maillages et des caméras. Elle est responsable de la gestion de l'affichage et de l'interaction avec les objets.
- La classe `Mesh` représente un maillage 3D et contient des informations sur les sommets, les faces et les arêtes. Elle est responsable de la gestion des données géométriques et de la génération des structures de données nécessaires pour l'affichage.
- La classe `Halfedge` représente une structure de données pour les maillages, permettant une navigation efficace entre les sommets, les arêtes et les faces. Elle est utilisée pour faciliter les opérations de modification et d'affichage des maillages.

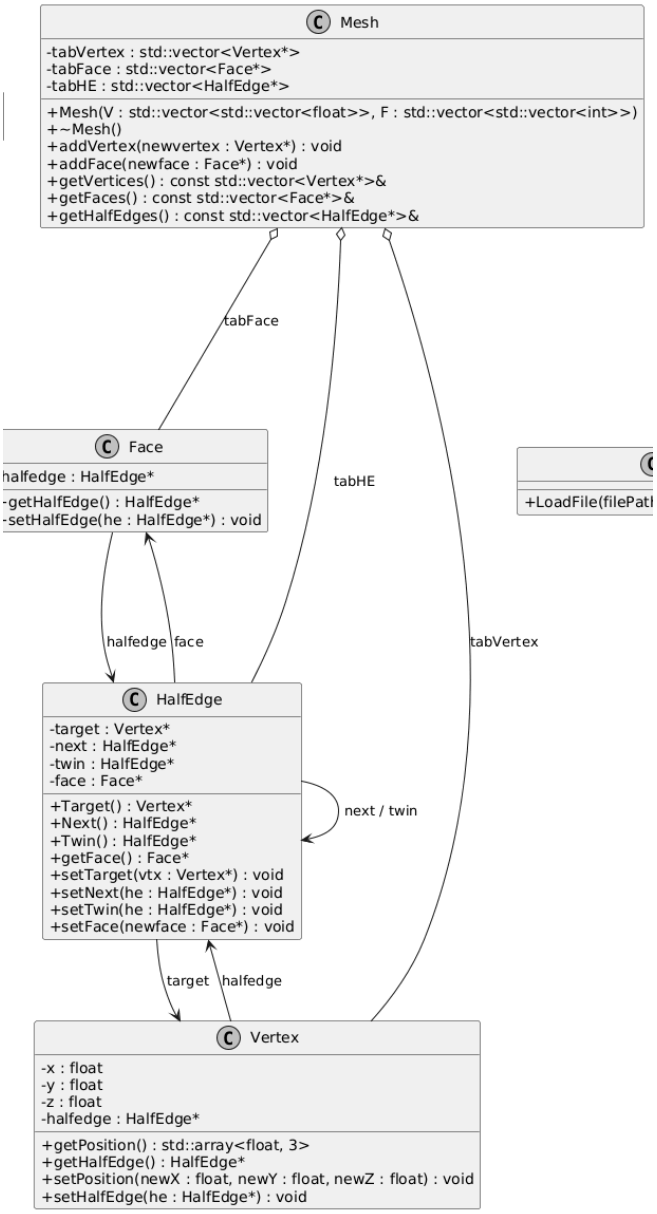


FIGURE 2 – Diagramme UML de la structure Mesh

Dans cette partie, nous voyons le diagramme UML de la structure du Mesh, qui représente un maillage 3D. La classe `Mesh` contient des attributs pour stocker les sommets, les faces et les arêtes du maillage, ainsi que des méthodes pour gérer ces éléments. Les relations entre les classes sont également représentées, montrant comment elles interagissent pour former la structure globale du projet.

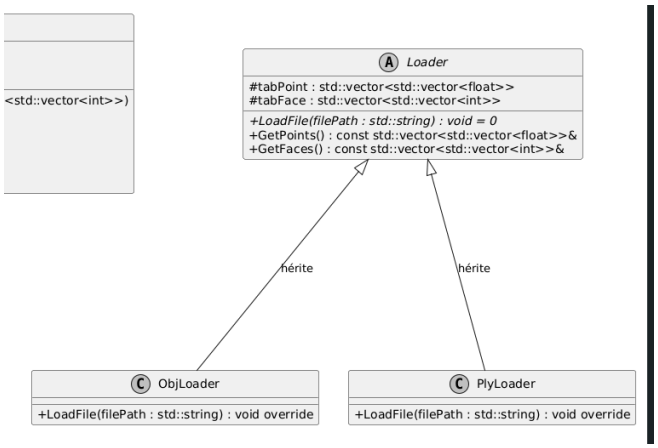


FIGURE 3 – Diagramme UML du Loader

Pour le loader, nous avons utilisé une classe générique pour permettre le chargement de différents formats de fichiers. Grâce à cela, nous avons pu créer des classes spécifiques pour chaque format de fichier, telles que `PLYLoader` et `OBJLoader`, qui héritent de la classe générique `Loader`. Cela nous a permis de gérer facilement l'ajout de nouveaux formats de fichiers à l'avenir.

5. Loader

Le système de chargement des fichiers de notre projet repose actuellement sur deux classes spécifiques, `PlyLoader` et `ObjLoader`, qui héritent toutes deux de la classe générique `Loader`.

La classe `PlyLoader` permet de charger les fichiers au format `.ply`, qu'ils soient encodés en ASCII ou en binaire au format *Little Endian*. Elle commence par lire l'en-tête du fichier afin d'en déterminer le format, le nombre de sommets, le nombre de faces, les types de données utilisés (`double`, `float`, `int`), ainsi que la présence éventuelle de propriétés supplémentaires, telles que la couleur ou l'opacité. L'implémentation de ce chargeur a notamment nécessité la gestion des octets à ignorer lors de la lecture des données binaires, par exemple ceux correspondant au remplissage (*padding*) ou à des propriétés qui ne sont pas exploitées par le programme. Nous avons utilisé des outils d'intelligence artificielle pour nous aider à mettre en œuvre cette gestion et à découvrir des outils C++ adaptés, comme `std::istringstream`, qui facilite l'analyse des données au format ASCII.

La classe `ObjLoader` permet de charger les fichiers au format `.obj`. Elle analyse chaque ligne du fichier en fonction de son préfixe afin de distinguer les sommets, identifiés par `v`, des faces, identifiées par `f`. L'utilisation de `std::istringstream` permet de séparer et d'extraire facilement les différentes valeurs présentes sur chaque ligne.

Enfin, les deux classes stockent les données extraites dans deux tableaux : `tabPoint` et `tabFace`. Le premier contient les coordonnées des sommets sous la forme d'un `std::vector<Eigen::Vector3d>`, tandis que le second contient les faces sous la forme d'un `std::vector<std::vector<int»`. Cette organisation permet de représenter les sommets et la connectivité du modèle 3D dans des structures de données directement exploitables par le reste du programme.

6. Halfedge

Notre structure Halfedge repose sur trois classes étroitement liées : `Vertex`, `Face` et `HalfEdge`.

La classe `Vertex` représente les sommets du maillage. Chaque sommet possède notamment une position, stockée sous la forme d'un `Eigen::Vector3d`, ainsi qu'un pointeur vers un objet `HalfEdge*` qui lui est associé.

La classe `Face` représente les faces triangulaires du maillage. Elle contient un pointeur vers l'un des objets `HalfEdge` qui la constituent, de type `HalfEdge*`. Les autres objets `HalfEdge` de la face peuvent ensuite être retrouvés en parcourant les liens entre eux.

La classe `HalfEdge` constitue le cœur de cette structure de données. Chaque objet `HalfEdge` contient plusieurs informations permettant de décrire sa position dans le maillage : un pointeur vers le sommet cible (*target*), de type `Vertex*`, un pointeur vers l'object `HalfEdge` suivant (*next*), un pointeur vers son opposé (*twin*), ainsi qu'un pointeur vers la face à laquelle il appartient (`Face*`).

L'organisation de ces liens permet de parcourir efficacement le maillage et d'accéder aux éléments voisins. Elle vise également à faciliter les opérations de modification de la géométrie et de la topologie du maillage. Cependant, ces fonctionnalités ne sont pas encore implémentées dans le projet.

La structure du maillage est générée par la classe `Mesh`, à partir des tableaux de sommets et de faces fournis par le module de chargement (`Loader`). Les éléments sont ensuite organisés dans trois tableaux : `tabVertex`, `tabFace` et `tabHE`, qui contiennent respectivement les sommets, les faces et les objets `HalfEdge` du maillage.

7. Affichage

Pour l'affichage, nous avons utilisé la bibliothèque `OpenGL`, qui est une API graphique puissante et flexible pour le rendu 2D et 3D. OpenGL nous permet de créer des scènes 3D interactives et de gérer les transformations, l'éclairage et les textures.

Voici les sommets d'un cube que nous avons utilisé pour tester l'affichage dans notre éditeur 3D :

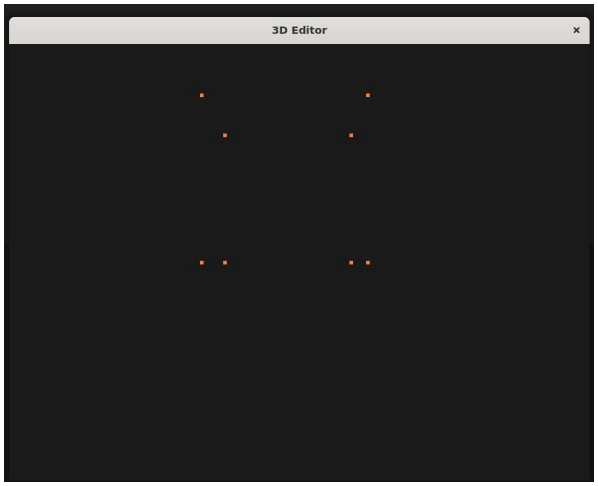


FIGURE 4 – Sommets d'un cube pour le test d'affichage

Ainsi que les faces triangulaires de ce dernier :

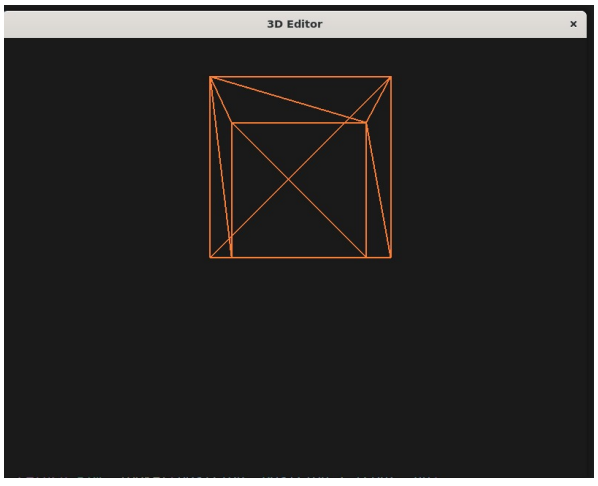


FIGURE 5 – Faces triangulaires d'un cube pour le test d'affichage

Par rapport au déplacement de la caméra, seuls les déplacements grâce aux touches ZQSD sont implémentés pour le moment. Le déplacement de la caméra est réalisé en modifiant la position de celle-ci dans l'espace 3D en fonction des entrées de l'utilisateur.

Nous avons également prévu d'ajouter des fonctionnalités supplémentaires, telles que le zoom et la rotation de la caméra, pour améliorer l'expérience utilisateur.

8. Conclusion

La majorité des bases structurelles du projet a été mise en place. Cependant, plusieurs fonctionnalités restent à développer afin d'atteindre les objectifs fixés.

La principale difficulté rencontrée au cours du projet est notre manque d'expérience en C++, notamment dans l'utilisation d'OpenGL, ainsi que notre connaissance encore limitée des différentes fonctions proposées par les bibliothèques que nous utilisons. Cela nous a demandé du temps pour comprendre leur fonctionnement et les utiliser correctement.

Plusieurs fonctionnalités restent à implémenter pour la suite du projet, comme la modification de la géométrie du modèle chargé, le calcul du plus court chemin entre deux sommets quelconques, l'optimisation des performances du programme, ainsi qu'une meilleure gestion des erreurs et des éventuels plantages.